

Báo cáo kết quả — Hệ thống quản lý thuê nhà bằng smart contract

1. Tóm tắt

Đề tài xây dựng một hệ thống quản lý thuê nhà chạy trên smart contract Ethereum, tự động hoá việc giữ tiền cọc, thu tiền thuê định kỳ, xác nhận bàn giao và tắt toán hợp đồng — thay thế cho việc phải "tin tưởng" một bên trung gian là con người. Toàn bộ 8 chức năng tối thiểu **và cả 3 chức năng nâng cao** theo đề bài đã hoàn thành, có test tự động (32 test); hệ thống đã deploy thật lên **Sepolia testnet**, có giao diện web kết nối MetaMask, có backend index dữ liệu on-chain, và đã qua một vòng rà soát bảo mật. Ngoài 11 chức năng theo đề bài, có thêm `cancelListing` — cho phép chủ nhà huỷ một tin đăng nhằm (vd sai giá) trước khi có người đặt cọc, giải quyết hạn chế thực tế của blockchain là dữ liệu **không sửa được sau khi ghi** (xem mục 4.3).

2. Mục tiêu & phạm vi

Xem chi tiết bài toán, actor, và phạm vi chức năng tại [phan-tich-bai-toan.md](#). Tóm tắt nhanh:

- Chức năng tối thiểu (8/8 đã hoàn thành):** đăng tài sản, đặt cọc (đồng thời tạo hợp đồng), thanh toán định kỳ, xác nhận bàn giao, hoàn/khấu trừ cọc, kết thúc hợp đồng, theo dõi lịch sử thanh toán.
- Chức năng nâng cao (3/3 đã hoàn thành):** cơ chế trọng tài khi tranh chấp (đề xuất/đồng ý/khiếu nại/trọng tài bỏ phiếu), phạt thanh toán trễ (5% nếu quá hạn), và multisig cho giải ngân tiền cọc tranh chấp (cần ≥ 2 trọng tài đồng thuận cùng 1 mức) — xem [gioi-han-va-rui-ro.md](#) § 3.

3. Kiến trúc & công nghệ

Lớp	Công nghệ	Ghi chú
Smart contract	Solidity 0.8.24, OpenZeppelin 5.x (<code>ERC721</code> , <code>AccessControl</code> , <code>ReentrancyGuard</code>), biên dịch <code>viaIR</code>	2 contract: <code>RentalAgreementToken</code> (token) + <code>RentalManager</code> (nghiệp vụ + trọng tài)
Môi trường dev	Hardhat 3 + TypeScript (ESM), Hardhat Ignition (deploy), Hardhat keystore (bí mật)	Bám theo bộ công cụ trong tài liệu quy trình mẫu
Blockchain	Hardhat Local Network + Ethereum Sepolia testnet (đã deploy thật)	Địa chỉ deploy hiện tại xem <code>frontend/src/config.js</code> hoặc <code>ignition/deployments/chain-11155111/</code>
Frontend	ReactJS + Vite, ethers.js v6, MetaMask	Tách UI khỏi nguồn dữ liệu qua lớp <code>RentalService</code> (interface chung cho <code>MockRentalService</code> / <code>ChainRentalService</code>); đã test thật với 2 ví MetaMask trên 2 trình duyệt (chủ nhà/người thuê)
Backend	Node.js + TypeScript, Express, <code>node:sqlite</code>	Event indexer + REST API, không phải nguồn dữ liệu chính — chỉ là lớp chỉ mục để tra cứu nhanh

Lớp	Công nghệ	Ghi chú
Lưu tệp	IPFS (<code>Property.imageCID</code>)	Ảnh phòng (không bắt buộc, chọn được nhiều ảnh) tham chiếu bằng danh sách CID cách nhau dấu phẩy; upload trực tiếp qua Pinata nếu người dùng tự cấu hình <code>VITE_PINATA_JWT</code> , hoặc dán CID đã upload sẵn
Test	Hardhat Test + Chai (Mocha)	32 test case, bao gồm time-travel test cho phạt trễ hạn
Bảo mật	Slither (gặp giới hạn tương thích) + rà soát thủ công	Xem mục 6
Quản lý mã nguồn	Git/GitHub	Đã push, repo công khai/riêng tư trên GitHub
Triển khai frontend	Vercel	Tự động deploy lại khi push lên <code>main</code>

Sơ đồ kiến trúc, sequence diagram chi tiết: [kien-truc-va-usecase.md](#). Vì sao chọn ERC-721 không chuyển nhượng và tách 2 contract: [lua-chon-token.md](#). Cấu trúc struct/enum/event của contract: [thiet-ke-du-lieu.md](#).

3.1. Vì sao tách token (`RentalAgreementToken`) khởi nghiệp vụ (`RentalManager`)

Mỗi lượt thuê thành công (`rentProperty`) mint một token ERC-721 **không chuyển nhượng** cho người thuê, đại diện cho quyền của hợp đồng thuê đó (`tokenId == propertyId`). Token chỉ ghi nhận "ai đang giữ quyền", còn toàn bộ luật nghiệp vụ (đặt cọc bao nhiêu, ai được trả tiền, khấu trừ thế nào, ai là trọng tài) vẫn nằm trong `RentalManager` .

4. Chức năng đã triển khai

4.1. Chức năng tối thiểu (8/8)

#	Chức năng	Hàm smart contract	Trạng thái
1	Chủ nhà đăng thông tin tài sản	<code>listProperty</code>	✓
2	Người thuê đặt cọc	<code>rentProperty</code>	✓
3	Tạo hợp đồng thuê	Gộp vào <code>rentProperty</code> + mint token đại diện	✓
4	Thanh toán định kỳ	<code>payRent</code> (tự tính phạt trễ nếu quá hạn)	✓
5	Xác nhận bàn giao	<code>confirmHandover</code>	✓
6	Hoàn/khấu trừ tiền đặt cọc	<code>proposeSettlement</code> → <code>acceptSettlement</code> / <code>voteOnDispute</code>	✓
7	Kết thúc hợp đồng	<code>_settle</code> (nội bộ, gọi từ bước 6)	✓
8	Theo dõi lịch sử thanh toán	Đọc event trực tiếp từ blockchain (tab "Lịch sử", bấm vào từng dòng xem chi tiết) + REST API backend	✓

4.2. Chức năng nâng cao (3/3)

#	Chức năng	Hàm smart contract	Trạng thái
1	Cơ chế trọng tài khi tranh chấp	<code>disputeSettlement</code> , <code>voteOnDispute</code>	✓
2	Phạt thanh toán trễ	<code>payRent</code> (kiểm tra <code>nextDueDate</code> , cộng <code>lateFeeBps</code>)	✓
3	Multisig cho giải ngân tiền cọc	<code>voteOnDispute</code> (cần <code>arbiterApprovalsRequired</code> phiếu đồng thuận)	✓

Chi tiết thiết kế & đánh đổi: [gioi-han-va-rui-ro.md § 3](#).

4.3. Chức năng bổ sung: `cancelListing` (huỷ tin đăng nhảm)

Các trường của `Property` (giá, cọc, mô tả...) chỉ ghi được **một lần** trong `listProperty` — đúng bản chất bất biến của blockchain, nhưng phát sinh vấn đề thực tế: đăng nhảm giá thì tin đó nằm sai vĩnh viễn, kể cả chủ nhà cũng không tự thuê được tin của chính mình để "dọn" nó đi (`rentProperty` chặn `msg.sender == p.landlord`). `cancelListing(id)` cho phép đúng chủ nhà huỷ tin **khi còn Listed** (chưa ai đặt cọc), chuyển sang trạng thái `Cancelled` mới — không chuyển tiền, không ảnh hưởng các tin khác. Có 4 test riêng (chủ nhà huỷ được, người khác không huỷ được, không huỷ được tin đã có người thuê, tin đã huỷ không thuê được nữa).

5. Kết quả kiểm thử

Chạy `npx hardhat test` — **32/32 test pass**, bao gồm:

- Nghệp vụ cơ bản: đăng tin, đặt cọc (đúng/sai số tiền), chặn chủ nhà tự thuê, tiền cọc giữ ở contract, trả tiền đúng hạn.
- Phạt trễ hạn**: dùng `networkHelpers.time.increase()` để mô phỏng quá hạn, xác nhận bắt buộc trả kèm phạt và hạn kỳ tiếp theo tính từ hạn cũ (không phải từ lúc trả trễ).
- Đề xuất/đồng ý/khiếu nại/trọng tài (multisig)**: chỉ chủ nhà đề xuất được, chặn đề xuất trước khi bàn giao, chặn khấu trừ vượt cọc, người thuê đồng ý → tắt toán ngay, người ngoài không tự vote được (`AccessControlUnauthorizedAccount`), 1 trọng tài chưa đủ ngưỡng, **2 trọng tài đồng thuận cùng mức → tự động tắt toán**, 2 trọng tài khác mức không tự tắt toán, trọng tài không vote 2 lần được.
- Token đại diện hợp đồng: mint đúng người, không chuyển nhượng được (cả 2 overload `safeTransferFrom`), tồn tại sau khi kết thúc.
- Bảo mật: chống reentrancy khi mint token trong `rentProperty` .
- Huỷ tin đăng (`cancelListing`)**: chủ nhà huỷ được tin còn trống, người khác không huỷ được, không huỷ được tin đã có người thuê, tin đã huỷ không cho thuê lại được.

6. Bảo mật

- Static analysis (Slither)**: đã cài đặt nhưng **không chạy được tự động** trên project này do `cryptic-compile` chưa hỗ trợ định dạng build-info của Hardhat 3, và bản thân `solc` (native, cài qua `solc-select`) lỗi resolve import trên đường dẫn có ký tự Unicode của thư mục project. Chi tiết lỗi và hướng khắc phục: [bao-mat-slither.md § 2](#).
- Rà soát thủ công thay thế**: rà soát theo đúng nhóm lỗi Slither thường phát hiện (reentrancy, access control, unchecked call, zero-address, gas/DoS), phân loại rõ Confirmed / Design concern / False positive / Accepted risk — xem [bao-mat-slither.md § 3](#). Không phát hiện lỗi nghiêm trọng có thể gây mất tiền/khoá tiền.

- **Biện pháp đã áp dụng trong code:** `ReentrancyGuard` + checks-effects-interactions cho mọi hàm chuyển ETH, kiểm tra giá trị trả về của mọi low-level `.call{value:}`, kiểm tra địa chỉ `0x0` ở constructor, token không chuyển nhượng, multisig N-trong-M cho quyết định trọng tài (1 trọng tài không tự quyết được). Bảng đầy đủ: [gioi-han-va-rui-ro.md § 2](#).

7. Kết quả triển khai (deployment)

- **Local (Hardhat Network):** đã deploy và demo thành công end-to-end nhiều lần trong quá trình phát triển (`npx hardhat node` + `npm run deploy` + backend indexer/API + frontend).
- **Sepolia testnet: đã deploy thật** bằng `npm run deploy:sepolia` (Hardhat Ignition, dùng Hardhat keystore cho `SEPOLIA_RPC_URL` / `SEPOLIA_PRIVATE_KEY` — không lộ khoá riêng tư ở đâu trong code/chat). Frontend deploy trên Vercel đã trở thẳng vào contract Sepolia này (không còn dùng chế độ dữ liệu mẫu). Đã kiểm thử thật bằng 2 ví MetaMask trên 2 trình duyệt khác nhau đóng vai chủ nhà/người thuê — xem kịch bản chi tiết tại [README.md § Kịch bản demo](#).
- **Đã deploy lại Sepolia nhiều lần trong quá trình phát triển** (thêm chức năng nâng cao trọng tài/phạt trễ/multisig, rồi thêm `cancelListing`) — mỗi lần contract đổi mã nguồn (khác constructor hoặc không) đều cần deploy lại vì contract không có proxy/cơ chế nâng cấp, địa chỉ contract vì vậy cũng đổi theo mỗi lần. Đã viết `scripts/relist-from-old-contract.ts` để tự động đăng lại các tin còn ở trạng thái `Listed` từ contract cũ sang contract mới (giảm thời gian đăng lại thủ công), và `scripts/grant-arbiter.ts` / `scripts/revoke-arbiter.ts` để quản lý `ARBITER_ROLE` sau mỗi lần deploy.

8. Mã nguồn & cấu trúc thư mục

Toàn bộ mã nguồn nằm tại thư mục project (`contracts/` , `test/` , `ignition/` , `backend/` , `frontend/` , `docs/`) — xem danh sách đầy đủ và mô tả từng phần trong [README.md](#) . Mã nguồn đã được đưa lên GitHub (bao gồm cả hồ sơ deploy Sepolia tại `ignition/deployments/chain-11155111/` để version-control địa chỉ contract đã deploy).

9. Giới hạn còn tồn tại

Xem đầy đủ tại [gioi-han-va-rui-ro.md](#) và [production-checklist.md](#). Tóm tắt các điểm chính:

- Quyết định trọng tài thực thi ngay khi đủ phiếu, không có time lock để bên còn lại phản ứng nếu trọng tài thông đồng.
- `ARBITER_ROLE` do admin (ví deploy) tự chỉ định — rủi ro tập trung quyền lực nếu khoá admin bị lộ.
- **Contract không tự chặn xung đột lợi ích khi bỏ phiếu trọng tài:** `voteOnDispute` chỉ kiểm tra `ARBITER_ROLE` , không kiểm tra người bỏ phiếu có phải chính chủ nhà/người thuê của tranh chấp đó không. Vấn đề này thực sự xảy ra khi test (ví admin/chủ nhà tự vote được cho tranh chấp của mình, vì admin mặc định có `ARBITER_ROLE`) — đã giảm thiểu bằng quản trị vai trò (thu hồi quyền khỏi ví admin, cấp cho ví độc lập), chưa sửa tận gốc ở contract. Chi tiết: [gioi-han-va-rui-ro.md § 4](#).
- Chỉ hỗ trợ thanh toán bằng ETH.
- Chưa có cơ chế tạm dừng khẩn cấp (`Pausable`), chưa có multisig cho ví admin.
- Slither không chạy tự động được (xem mục 6); chưa có audit độc lập từ bên ngoài; chưa có fuzz/invariant test (Foundry).
- Backend dùng SQLite (đủ cho demo/đồ án), chưa phù hợp production nhiều người dùng đồng thời.

10. Kết luận

Hệ thống đáp ứng đầy đủ **8/8 chức năng tối thiểu** và **3/3 chức năng nâng cao** theo đề bài (cộng thêm `cancelListing` ngoài yêu cầu), có kiến trúc tách UI/BE rõ ràng, có bộ test tự động bao phủ cả nghiệp vụ lẫn bảo mật (32 test), đã **deploy thật lên Sepolia** và kiểm thử bằng ví thật trên nhiều trình duyệt/tài khoản, và đã được rà soát bảo mật (dù công cụ tự động Slither gặp giới hạn kỹ thuật, đã có rà soát thủ công thay thế và ghi nhận minh bạch). Các giới hạn còn lại (time lock trọng tài, multisig admin, xung đột lợi ích khi bỏ phiếu chưa chặn ở contract, audit độc lập...) đã được liệt kê trung thực, không tick giả.
